Microprocessors and Microsystems 94 (2022) 104634

Contents lists available at ScienceDirect

Microprocessors and Microsystems

journal homepage: www.elsevier.com/locate/micpro

# A novel framework for deployment of CNN models using post-training quantization on microcontroller

Sailesh M [a], Selvakumar K [b,*], Narayanan Prasanth [c,*]

[a] School of Electrical Engineering, Vellore Institute of Technology Vellore, Tamilnadu, India
[b] Department of Instrumentation, School of Electrical Engineering, Vellore Institute of Technology Vellore, Tamilnadu, India
[c] Department of Database Systems, School of Computer Science and Engineering, Vellore Institute of Technology Vellore, Tamilnadu, India

ARTICLE INFO

*Keywords:*
Machine learning
Convolution neural network
Quantization
Voice Recognition
TinyML
Fixed point

ABSTRACT

Nowadays, microcontrollers are very common in a wide range of applications. Integrating Machine Learning with Embedded Systems is essential and should be provided at a low cost to satisfy modern-day requirements. The tools that currently exist for this purpose can be used only with the hardware provided by the same vendor, and are usually not flexible with IDEs. This paper proposes an open-source framework that helps to deploy a trained Neural Network (Both Convolution and Deep Neural Network) on any ARM Cortex M-based microcontroller, providing an easy and low-cost device. It converts the trained model with *float32* parameters into *int8* type parameters and automatically generates inference code using SIMD optimized CMSIS functions. The output obtained using the proposed framework is validated with the X-Cube-AI framework. Also, A voice-based digit classification system is implemented using a Cortex-M4 microcontroller to demonstrate the efficiency of this procedure.

## 1. Introduction

TinyML focuses on implementing sophisticated deep learning models embedded in the same microcontroller, which has the logic to manage the connected sensors and actuators. Typically, Machine Learning (ML) models are deployed in one of the platforms described in Fig. 1. GPUs and FPGAs have high computation power, and are often used in the Cloud or edge platforms to perform inference [1]. However, this in turn could cause a high latency and make the system dependent on network connection. Unlike leveraging the cloud technology, inference done at the microcontroller offers significant advantage when implemented in industrial scenario and has gained traction in recent years. Since this approach does not depend on external resources, latency, network dependency, security and privacy challenges can be avoided, thus facilitating an efficient, cost-effective strategy [1]. Also, in many resource limited applications like industrial machinery fault diagnosis [2], human activity tracking [3], wildlife monitoring [4], and aerial robotics [5], microcontrollers are widely adopted. This creates a necessity to find a way to deploy ML models on a microcontroller.

Over the last few years, applications such as spoken keyword spotting [6] and Human Activity Recognition [3] have been implemented in real-time on 32-bit low-power microcontrollers. After the network has been trained, it is deployed onto a microcontroller to perform the inference on the target platform. According to [13], deployment of an ML model involves the following phases:

- Extraction of weights and biases of a given model and encoding to desired precision suitable for on-device inference,
- Development of inference program according to the given model graph,
- Compilation and flashing of the inference program onto the microcontroller's flash memory.

The main challenges faced when deploying ML in microcontrollers are limited hardware resources in terms of memory and computation speed and lack of a unified TinyML framework for embedded devices [7, 8]. Most models have floating-point parameters which requires huge memory and computation time. This issue is addressed by converting *float32* into *int8* models using post-training quantization techniques [9, 10], and [11]. [12], deployed a Binary Neural Network (BNN) architecture for sound event detection in a RISC-V core GAP8 microcontroller. It is reported that BNN offers 10x better computation efficiency when compared to the performance of ARM Cortex-M7 *int16* implementation at the cost of 7.3% lower accuracy. Unfortunately, the

* Corresponding authors.
*E-mail addresses:* sailesh.m2018@vitstudent.ac.in (S. M), selvakumar.k@vit.ac.in (S. K), n.prasanth@vit.ac.in (N. Prasanth).

https://doi.org/10.1016/j.micpro.2022.104634
Received 26 April 2022; Received in revised form 15 July 2022; Accepted 17 August 2022
Available online 22 August 2022
0141-9331/© 2022 Elsevier B.V. All rights reserved.

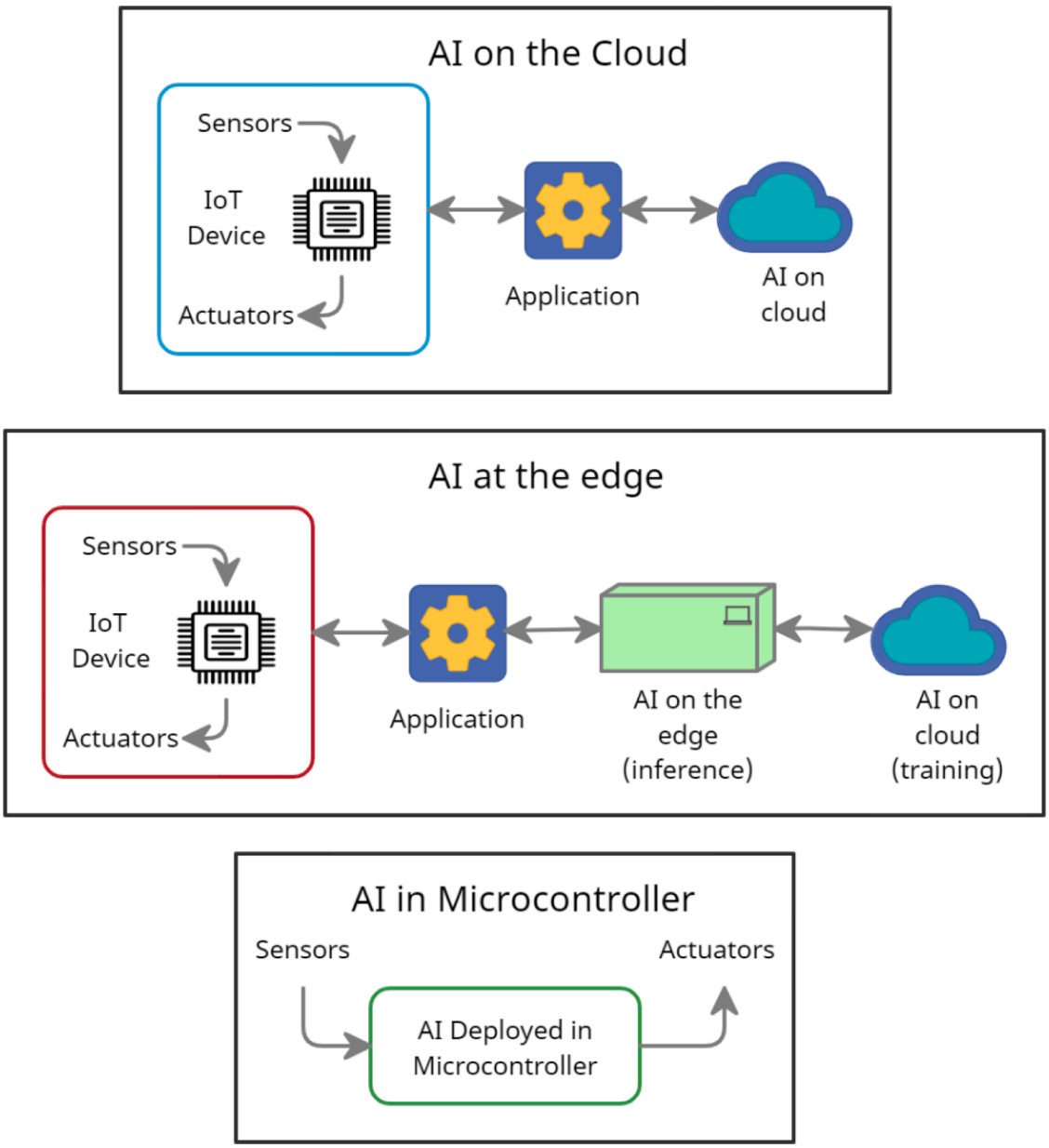



**Fig. 1.** ML Deployment Strategies.

microcontrollers implementing open RISC-V ISA are still scarce in the market. Implementing BNN in a microcontroller requires manually packing and unpacking data, introducing a computation overhead even though it helps reduce the memory footprint

For implementing post training quantization, numerous well-established frameworks like TensorFlow Lite, Qualcomm AIMET, Microsoft NNI, Intel OpenVINO and Nvidia TensorRT [8] are available to generate model libraries that can be used as APIs during runtime. However, these tools are available to generate quantized runtime engines only to higher end edge devices like mobile phones, single board computers not for microcontrollers. A typical microcontroller has 256 KB RAM and 1 MB of flash, while a mobile phone can have 4 GB of RAM and 64 GB of storage [8].

At the time of this study, there are several frameworks available to support different microcontroller platforms, namely TensorFlow Lite for Microcontrollers (TFLM) [14], X-Cube-AI ([15], User manual), Texas Instruments Deep Learning, etc. The major issue with TFLM is, it generates C++ based inference library, but most embedded developers are comfortable in C. Moreover, debugging becomes very complex while building any customized application in specific IDEs. A developer should be able to make changes to the model without relying on code generators. On the other hand, X-Cube-AI's inference library is entirely proprietary/closed-source and it is not possible to manipulate or extend it. A major drawback of currently available tools is that they implement the runtime library of a given model specifically for a hardware platform [16]. There is no vendor-independent unified framework to integrate the quantized models into convenient embedded software development IDEs.

As of today, microcontrollers used in most commercial embedded systems are primarily relying on ARM Cortex-M cores with the associated ISA. It supports (Single Instruction Multiple Data) SIMD for different byte-sized operations of neural network layers making it easy to deploy the 8-bit precision quantized models.

Finally, there are less literature describing end-to-end deployment and evaluation process for a particular application. To address all the mentioned issues, we have proposed an open-source framework with a complete pipeline for deployment of a quantized model on a Cortex-M microcontroller. The major contributions of this paper are detailed

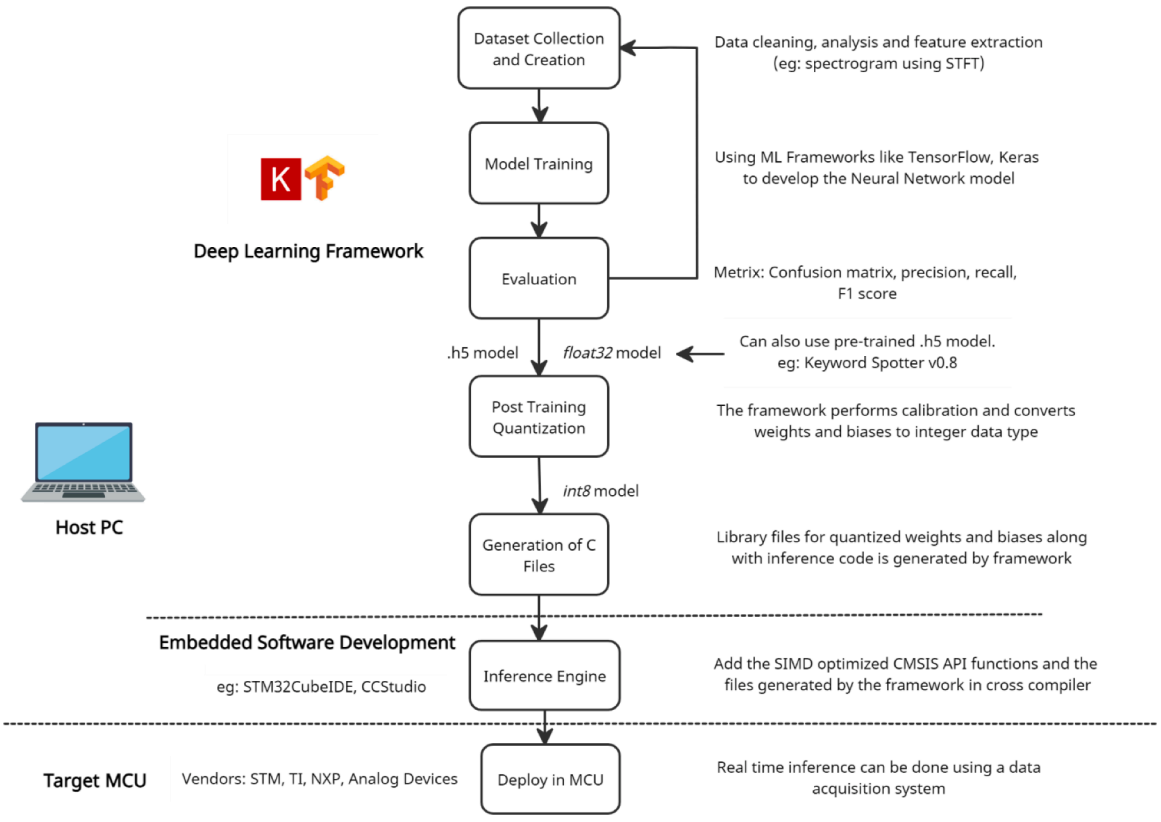



**Fig. 2.** Framework for deployment of *int8* quantized CNN model.

below:

1. Implementation of an open-source post training quantization and inference framework that automatically generates inference code from trained Neural Network model for Cortex-M microcontrollers.
2. Inference on 8-bit integers is accelerated using SIMD and other optimizations provided by the open source CMSIS-NN library from ARM.
3. Demonstrated end-to-end real-time deployment of quantized CNN model for voice-based classification of digits (0–9) using a Cortex-M4 microcontroller. The pipeline involves audio acquisition with a low-power microphone, spectrogram-based feature extraction, and on-board classification using the quantized CNN model.
4. The proposed framework has also been compared with X-Cube-AI in terms of memory consumption.

The rest of the paper is organized as follows: Section 2 presents the overview of the proposed open framework to deploy a quantized CNN model step-by-step. Section 3 presents the methodology to pre-process the recorded speech signals to obtain its spectrogram and train a CNN model. Section 4 describes the scheme for integer quantization of CNN models. Section 5 demonstrates a real-time voice-based digit classification system and Section 6 deals with the framework evaluation. Finally, Section 7 concludes the paper with the scope of the proposed work.

## 2. Proposed framework for deployment

Our proposed framework focuses on the most commonly utilized Neural Network algorithms. Fig. 2 details the steps involved in deploying a Neural Network onto a microcontroller.

The framework is implemented with the help of open-source libraries available in Python and can be easily used by everyone. The first step in training a CNN model requires obtaining quality data since it forms the backbone of any ML model. The algorithm's performance depends on the quality of the data fed to it during the learning phase. Therefore, most of the effort is focused on data extraction, cleaning, and analysis. Once the data quality is ensured, model development can be carried out followed by an evaluation of its accuracy. It should be noted that the model is trained with *float32* parameters at this stage.

**Algorithm 1**
- Model calibration.

Input : Number of images (k = 1,..., M), Number of layers ($i = 1$ ,.., N)
Output : Activation max $A^{k}_{max}$
1. *for* $k = 1, .., M$
2. *for* i $= 1$ , .., N
3. $A^{i,k}_{max} = \max(abs(A^{i,k}))$
4. end
5. $A^{k}_{max} = \max(A^{i,k}_{max})$
6. end

The different stages in deploying the trained CNN model on a microcontroller are listed below:

1. Input a CNN model built using any TensorFlow frame work and parse all model parameters (for example, number of layers, weights and biases of each layer, number of filters, etc.).
2. Obtain the scaling parameters using model calibration (Algorithms 1 and 2), quantize weights, biases to *int8* precision and generate library files for each layer.
3. Generate C code for inference engine using CMSIS-NN APIs.
4. Include the weights and bias library files along with the C code for inference in the custom application project of any IDE
5. Build, deploy and test the application in the microcontroller.

The proposed framework takes a pre-trained model as input, automating the process of quantizing the weights, biases, and generation of inference code, making it easier to deploy onto the microcontroller. To

**Algorithm 2**
- Obtaining Quantization Parameters.

Input : Test input X, parameter values $\{W_k, B_k, A_k\}$ of layer k, maximum value of activations $A_{max}$ and weights $W_{max}$,
Number of layers (i 1,…, k)

1. for i 1: k
2. ifi == 1
3. $T_i = \frac{2^N}{X_{max}}$
4. $Q_{a1} = 2^{(nextpow2(Ti)-1)}$
5. $A_{1,s8} = \| X \times Q_{a1} \|$
6. else }→*Input*Quantization
7. $T_i = \frac{2^N}{A_{max}}$
8. $Q_{ai} = 2^{(nextpow2(Ti)-1)}$
9. $A_{i,s8} = \| A_{i,out} \times \frac{Q_{ai}}{Q_{a(i-1)} * Q_{w(i-1)}} \|$
10. end
11. $Q_{Wi} = \frac{2^N}{W_{i,max}}$ }→*Weight*Quantization
12. $W_{i,s8} = \| W_i * Q_{Wi} \|$
13. $b_{i,s32} = \| B_i * Q_{Wi} * Q_{ai} \|$ →Biasquantization
14. Adapt CNN model, replacing $l_k$ with $Q(l_k)$
15. end

demonstrate the framework, we have implemented an end-to-end deployment for the voice digit classification system in the following sections.

## 3. Convolution neural network for digit classification

This section explains the bottlenecks in implementing a CNN model on an end device and methods for effective real-time execution for a voice-based digit classification CNN model. In addition, the steps involved in training a CNN model using recorded speech data for the classification of digits (0–9) are also discussed briefly.

### 3.1. Problem definition

Efficiency and precision are crucial factors that must be considered when implementing CNN models on end devices. Hence, converting the parameters from floating-point to fixed-point is necessary to deploy CNN models on end devices. This conversion process for obtaining fixed-point parameters from the trained CNN model is known as post-training quantization [17,18]. Generally, CNNs are highly capable of capturing high-level information from images and are used in classification and recognition. Its performance in visual processing has been quite promising that eventually led to achieving convincing results in audio recognition.

Speech-based Digit classification has been utilized to demonstrate the working of post-training quantization. The scope of voice-based command recognition is increasing rapidly due to the value it adds to commercial products. Implementation of CNN is chosen instead of DNN due to its superior spectral representations with time-frequency correlations [19]. For voice-based applications, on-device deployment of CNN inference is crucial for low power and mobile features. With the development of high processing hardware and the availability of public datasets, it becomes easier to train and implement a CNN in real-time. Google's Speech Command dataset is aimed at developing on-device CNN models that can be used to classify voice-based digit commands

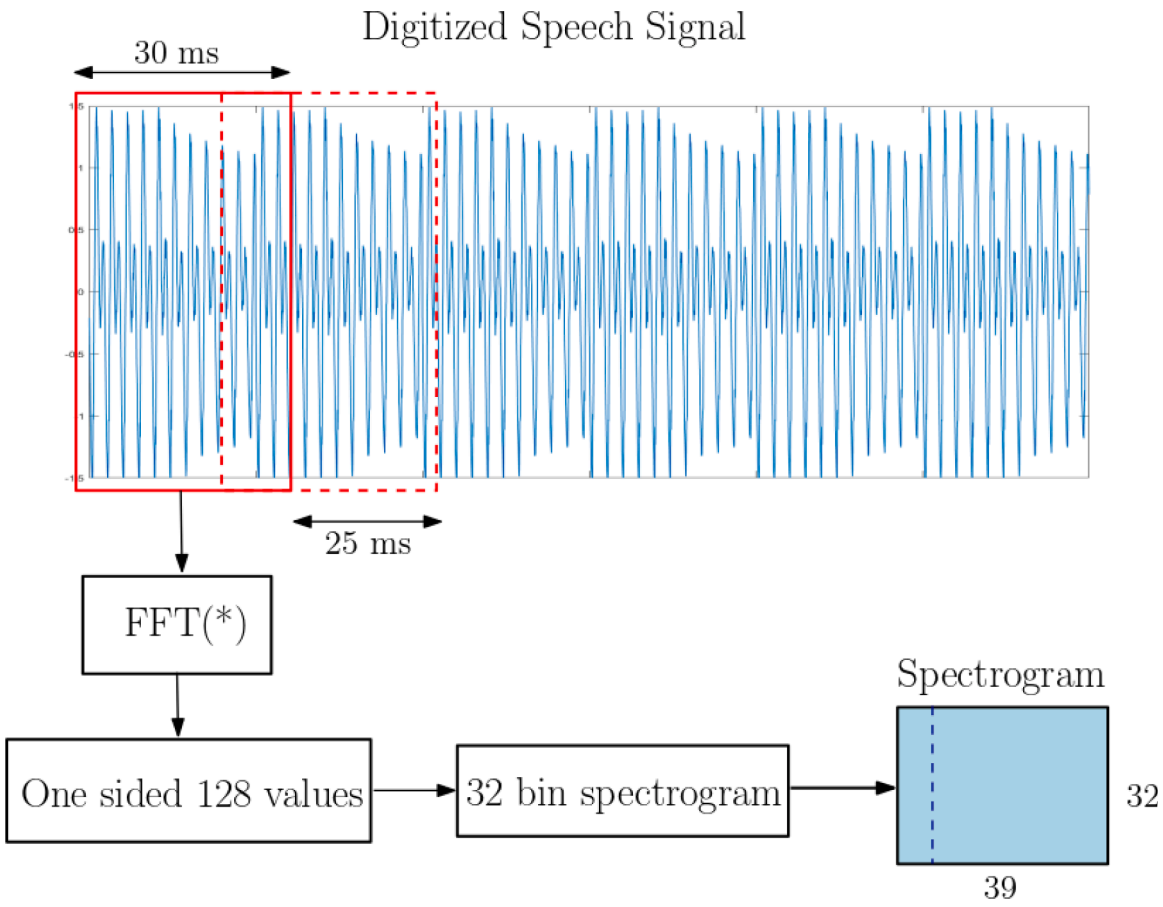



**Fig. 3.** Process of Spectrogram computation.

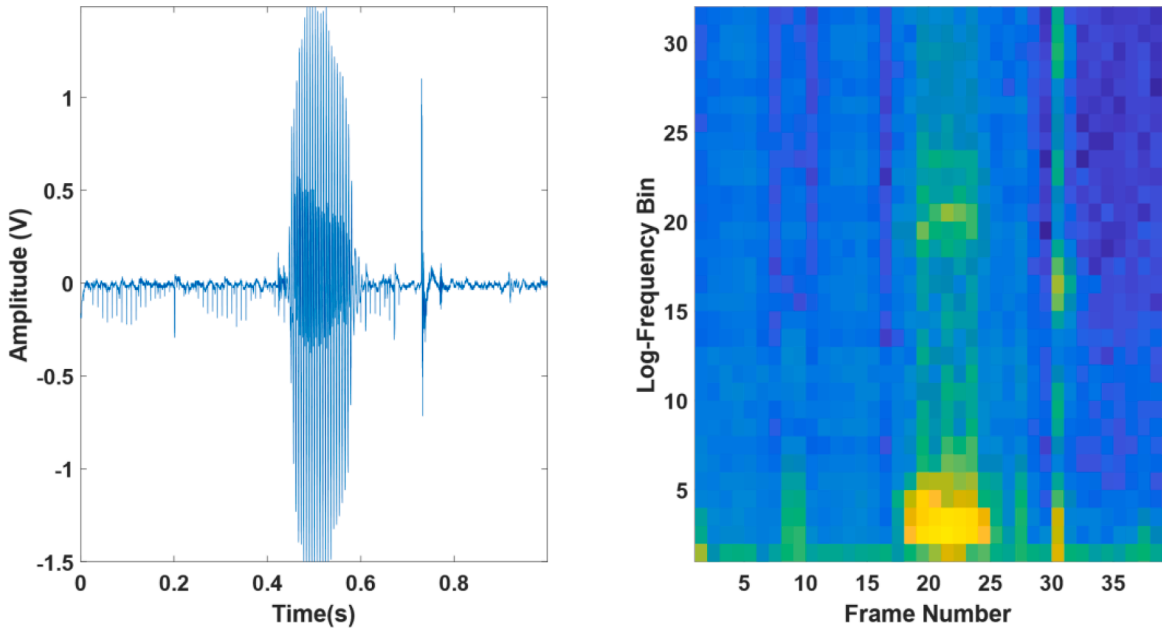



**Fig. 4.** a) Speech Command Dataset Signal 'FIVE' and, its (b) Log transformed Spectrogram.

[20]. The dataset contains 65,000 one-second audio clips of 30 short words, of which 12 occurrences are used in this paper. (http://download.tensorflow.org/data/speech_commands_v0.01.tar.gz). The audio recordings are sampled at 16,000 Hz. However, for our application, records are down-sampled by 2 due to resource constraints. The model is trained to classify the incoming audio into one of the 12 classes, 'zero', 'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'background', and 'unknown'.

The network is trained with spectrograms of audio clips which are the two-dimensional representation of audio frequency spectra over time. Once the CNN is trained using the dataset, the model is quantized and implemented in a microcontroller. Real-time implementation of

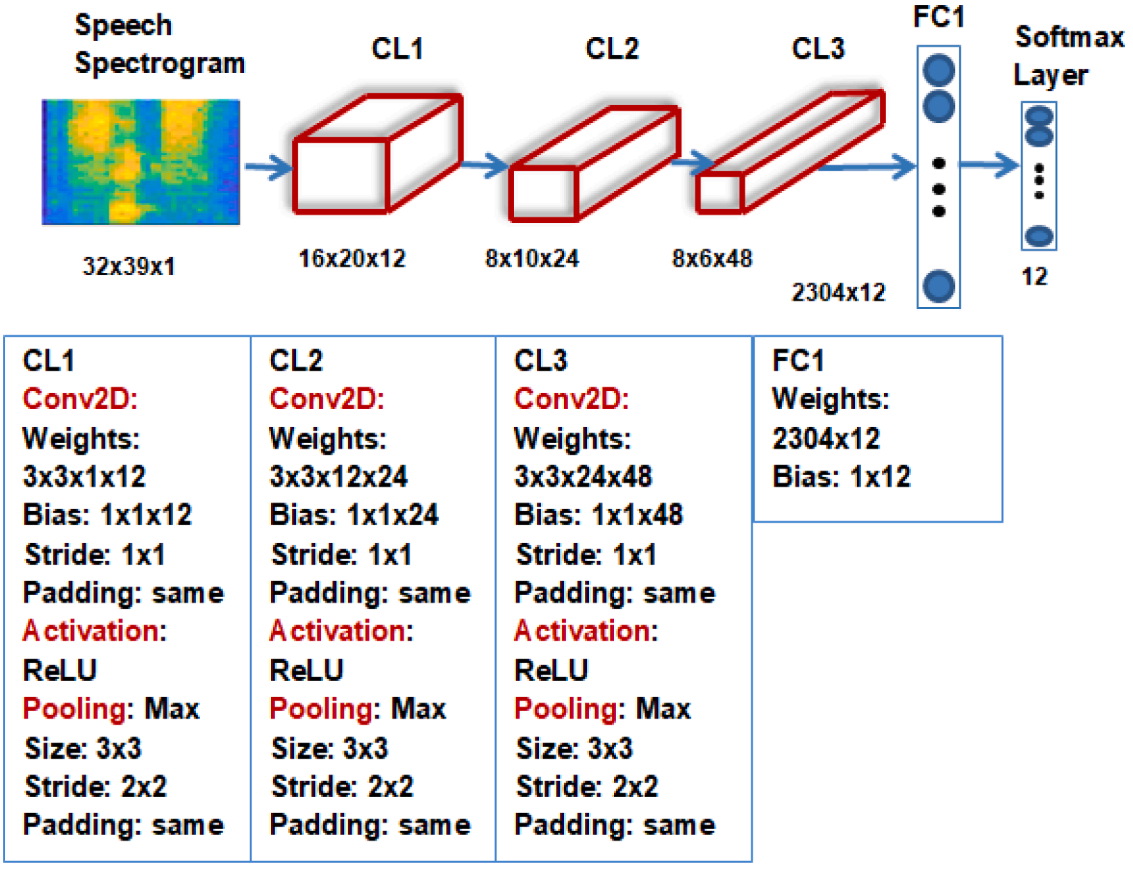



**Fig. 5.** CNN Model Architecture for speech digit classification.

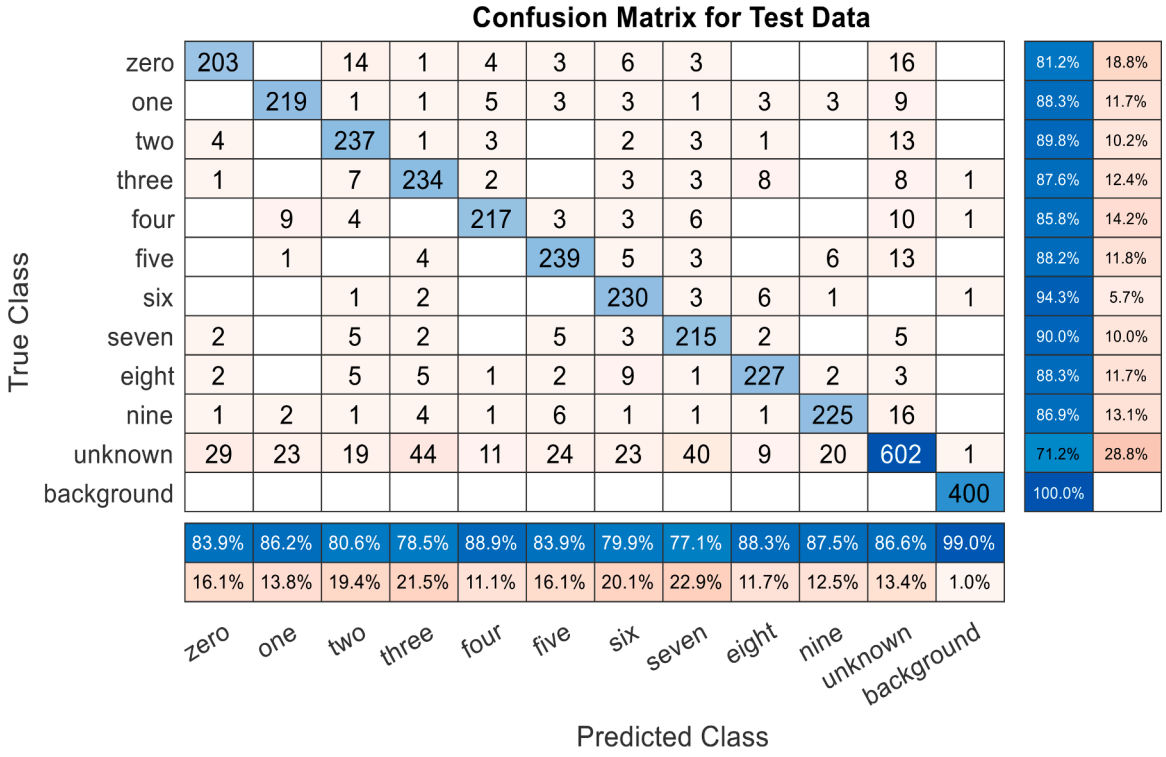



**Fig. 6.** Confusion Matrix for Test data.

speech recognition involves the following steps:

(a) Data acquisition,
(b) Data pre-processing, feature extraction and
(c) CNN inference using a pre-trained neural network.

### 3.2. Feature extraction

This section explains the Fast Fourier Transform (FFT) method-based feature extraction from short digital records using spectrograms. Though the Mel-scale spectrogram outperforms the linear scaled one, we have chosen the latter for simplicity at the cost of performance [21]. The audio signal is sampled at 8000 Hz as per the Nyquist-Shannon sampling theorem. The digital waveform acquired for one second is split into 39 frames of 30 milliseconds, with an overlap of 5 milliseconds as shown in Fig. 3. Since the sampling rate is 8000 Hz, each frame would contain 240 samples and 40 overlap samples. Each frame is transformed into its spectrogram representation. To compute the spectrogram, FFT is applied to each frame and this results in 256 entries of complex-valued output as given below:

$$STFT\{x_n\}(h,k) = X(h,k) = \sum_{n=0}^{N-1} x_{n+h} e^{\frac{-i2\pi kn}{N}},\ k,n = 0,\ldots,N-1 \quad (1)$$

where x is the input signal, N is the FFT length, and h is the hops' starting point. Since FFT is mirrored, only the first half of the magnitude spectrum (128 values) is utilized. Every sequence of four entries in the one-sided spectrum is averaged together, which gives a total of 32 frequency bins that results in a spectrogram image with dimensions of 32 × 39 as shown in Fig. 3.

To obtain data with a smoother distribution logarithm of the spectrogram is taken as $20\log_{10}X(h,k)$. A sample digital record 'FIVE' from the dataset and its log scaled spectrogram is shown in Fig. 4. (a) and (b) respectively. This extracted feature image is used to learn CNN based speech digit model.

**Table 1**
Speech digit classification model summary.

| Layer type | Learning parameters | *float32* size (KB) |
|---|---|---|
| Convolution layer 1 | 108 + 12 | 0.47 |
| Convolution layer 2 | 2592 + 24 | 10.22 |
| Convolution layer 3 | 10,368 + 48 | 40.69 |
| Fully connected layer C1 | 27,648 + 12 | 108.05 |
| Total | 40,812 | 159.43 |

### 3.3. CNN model architecture and training

The CNN architecture for the classification of digits based on speech input is shown in Fig. 5. As mentioned before, this model has been trained to classify the incoming audio recording into one of the 12 classes. Considering the primary focus is on model quantization, we have chosen our network architecture to be relatively simple, with only three convolutional layers and one fully-connected layer. In a typical CNN architecture, the convolutional layers are used along with ReLU based activation functions. Further, the max-pooling layers are used to reduce the feature dimensionality. Max pooling is also used to capture the time-translation invariance in the input spectrogram image ( [22]). The flattened features are given as input to the fully connected layer, followed by the SoftMax layer. As shown in Fig. 5, the output SoftMax layer reflects the probabilities associated with each pre-defined class. Adam optimization algorithm is selected to optimize the training of CNN, and cross-entropy is chosen as the loss function [23].

Using the TensorFlow framework, the model is trained for 25 epochs, with 220 iterations per epoch in a Python environment. Drop-out layers are used between each layer to avoid the model memorizing the specific features.

### 3.4. Model evaluation

The trained speech Digit model is evaluated against 3775 test audio clips from which the accuracy was computed to 86%. To summarize the model's performance against each class, the confusion matrix is plotted with precision and recall, as shown in Fig. 6. Diagonal elements

correspond to the accurately classified observations, while the off-diagonal elements represent incorrectly classified observations. It can be observed from Fig. 6 that the largest confusion value is between the 'unknown' and all other digits. However, higher accuracy can be achieved by increasing the network depth and the number of convolutional filters. Based on the model parameters, the memory required for storage (for *float32* data type) is given in Table 1. The total size of the network is approximately 160 kgbytes. When buffers for activations of each layer are included, high memory requirement becomes a challenge for deploying it in a microcontroller.

## 4. Post-training quantization

This section proposes a scheme to deploy the CNN inference engine in computationally efficient *int8* precision. The quantization scheme is detailed for converting a *float32* CNN model into an *int8* or *q7* precision fixed point model.

### 4.1. Fixed point representation

An introduction to fixed-point representation is necessary to understand its advantages over floating-point representation. The floating-point numbers are generally stored with one bit corresponding to the sign, and the remaining bits represent the mantissa and exponent parts. It is important to note that there is no fixed number of bits for the integer or the exponent parts. According to the IEEE 754 standard, some bits are also allocated for special values, like denoting infinity, zero, error, or representing a denormalized number. For fixed-point representation, as the name suggests, the number of bits allocated for integer and fractional parts is constant, and a separate bit is allocated for the sign.

In Q [m, n]-format, depending on where the binary point is assumed to be, a given number can be interpreted in different ranges [24]. In Q-format, m decides signed integer range and n decides the range of the fraction to be represented. For example, in *q7* or Q [1, 7] precision, 7 bits are used for fraction and 1 bit is used for the sign and the supported range of float values is $(-1, 1-2^{-7})$.

### 4.2. Integer quantization

For full integer quantization, the range (min, max) of activations of each layer in the model needs to be estimated. Since the model input and activations are variable, the model output cannot be calibrated unless a few inference cycles are simulated. As a result, a representative dataset is necessary to calibrate the weights and activations of the model. This dataset can be a small subset (around ~100–500 samples) of the training or validation data. Algorithm 1 presents the steps for offline calibration of the model to determine the range of activations (outputs of intermediate layers) for all the layers.

Once the range of activations for every layer is determined, the next step involves determining the maximum value of weight W in each layer of the model as shown in (2).

$$W_{\max} = (abs(W))_{\max} \tag{2}$$

For a N-bit quantization, the quantization factors for weights $Q_W$ and activations $Q_A$ are determined by (3).

$$\begin{aligned} Q_W &= \frac{2^{N-1}}{W_{\max}} \\ Q_A &= \frac{2^{N-1}}{A_{\max}} \end{aligned} \tag{3}$$

Finally, the quantized weights, bias, and activations are given by (4)

$$\begin{aligned} W_{s8} &= \| WQ_W \| \in \left(-2^N, 2^N - 1\right) \\ A_{s8} &= \| AQ_A \| \in \left(-2^N, 2^N - 1\right) \\ b_{s32} &= \| b * Q_W * Q_A \| \in \left(-2^{4N-1}, 2^{4N-1} - 1\right) \end{aligned} \tag{4}$$

The proposed scheme works for any publicly available lite weight models such as the Caffe model (https://caffe.berkeleyvision.org/), .h5 model from Keras framework (https://keras.io/), or trainedNet from MATLAB Deep Learning toolbox (https://in.mathworks.com/products/deep-learning.html), etc. The following steps are performed when quantizing parameters of a layer k from the given pre-trained CNN model to fixed-point representation.

1) From the CNN model, the parameter values $l_k \in \{W_k, B_k, A_k\}$ of layer k are extracted.
2) The extracted parameters are used to quantize the $l_k$ into fixed point representation, therefore producing $Q(l_k)$ which represents the quantized weights, bias and activations.
3) The quantized parameters are then used in the network. The quantized weights and biases perform the same functions as that of the floating-point parameters within the CNN. However, for activations, the network structure is altered by inserting a re-quantization step since the output of the convolution layer is in *int32* format. For convolutional layers, this step is carried out after the activation function (ReLU), while for the fully connected layer, re-quantization is computed before the activation function (Softmax) [9].
4) Steps 1,2,3 are repeated for every layer and finally inference is carried out using the quantized model. Test data is used to measure the accuracy of the quantized CNN which is then compared with the accuracy of the floating-point CNN.

Algorithm 2 illustrates the steps involved in obtaining quantization parameters for every layer k. The algorithm takes the parameters $\{W_k, B_k, A_k\}$ as input for every layer k along with test data X in order to quantize the parameters.

The function $\| . \|$ rounds the value to the closest integer. Values beyond the given range are to be saturated to maximum and minimum values.

In a convolution layer, 2-D convolution cum bias operation for any arbitrary element $a(i,j)$ in an input image with 3 * 3 filter weights(w) performed for $k^t$ channel is given in (5).

$$\begin{aligned} x_{(i,j)} &= w_{(0,0)}a_{(i-1,j-1)} + w_{(0,1)}a_{(i-1,j)} + w_{(0,2)}a_{(i-1,j+1)} + w_{(0,1)}a_{(i,j-1)} + w_{(1,1)}a_{(i,j)} \\ &+ w_{(1,2)}a_{(i,j+1)} + w_{(2,0)}a_{(i+1,j-1)} + w_{(2,1)}a_{(i+1,j)} + w_{(2,2)}a_{(i+1,j+1)} + b_k \end{aligned} \tag{5}$$

Where $x_{(i,j)}$ is output of convolution operation and $b_k$ is the bias value.

For fixed point computation, the model's weights and biases are quantized to 8- bits for all layers and all computations for each layer are carried out using 8-bits and accumulation in 32 bits as given by (6).

$$x_{s32} = W_{s8}A_{s8} + b_{s32} \approx Q_A Q_W \left(W_{f32}A_{f32} + b_{f32}\right) = Q_A Q_W x_{f32} \tag{6}$$

To trace the floating-point implementation, fixed point activations have to be scaled from the output of the convolution layer as given by (7).

$$x_{f32} \approx \frac{1}{Q_A Q_W} x_{s32} \tag{7}$$

Where, $\frac{1}{Q_A Q_W}$ represents the conversion factor.

To implement the entire network using *int8*, with appropriate $Q_A$ and $Q_W$ factors as the nearest power of 2 integers, right shifting operation can be used as given in (8) and (9):

$$W_{\max}{}' = 2^{\lceil \log_2 W_{\max} \rceil}, A_{\max}{}' = 2^{\lceil \log_2 (A_{\max}) \rceil} \tag{8}$$

$$Q_W{}' = \frac{2^{N-1}}{W_{A_{\max}}}, Q_A{}' = \frac{2^{N-1}}{A_{\max}{}'} \tag{9}$$

Since the output of each convolution layer is in *int32* format, further quantization is necessary to represent the output in *int8* format. This factor is determined in (10),

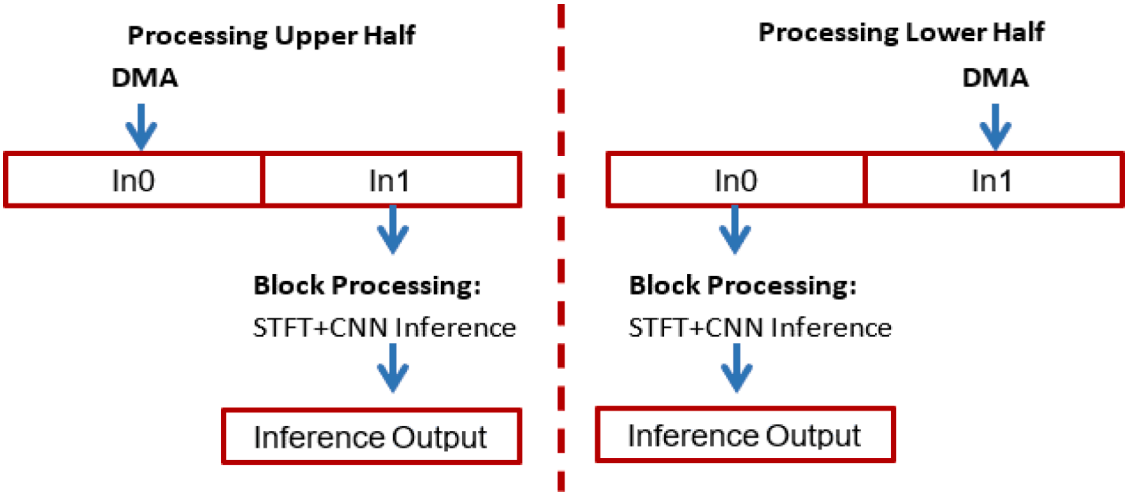



**Fig. 7.** Block Processing using Half and Full complete Interrupts.

$$A_{i,s8} = \left(\frac{Q_{A_i}{}'}{Q_{A_{i-1}}{}' Q_{W_{i-1}}{}'}\right) x_{s32} = Q_R x_{s32} \tag{10}$$

where $Q_R$ is the re-quantization factor and the right shift amount is determined using (11):

$$rs = \log_2 Q_R \tag{11}$$

Activations are dequantized using the right shift amount as given by (12):

$$A_{i,s8} = (int8)(x_{s32} >> rs) \tag{12}$$

While the quantization of input, filter kernels, and biases are done offline during the model translation stage, activation re-quantization is done during runtime [11]. Netron, an open-source neural network visualization tool, is used to verify the scaling parameters of the weights, biases, and activation in every layer [25]. The scaled parameters of the quantized model from the proposed method accurately match the parameters visualized through the open-source tool. The model can be implemented in real-time for classifying voice-based digits using the framework outlined.

## 5. Real time implementation

This section describes the steps involved in hardware implementation in detail such as data acquisition from the microphone, spectrogram computation, and CNN inference with parameters in *q7* format using an end device.

### 5.1. Hardware

Using simple and low-cost components, the speech digit classification system is implemented at around 25$. For speech acquisition, a low-cost electret microphone [26] with amplifier IC MAX4466 is interfaced with STM32F407 cortex M4 microcontroller, which has 1Mbytes of flash memory and 192Kbytes of RAM and has a maximum clock frequency of 168 MHz. SIMD optimized ARM-CMSIS NN library is used for CNN inference implementation.

The microphone needs a 3 V DC voltage supply and has an internal DC bias of VCC/2 (1.5 V). So, the microphone will output a steady 1.5 V when the test environment is quiet. The sensor gives an output that is in both positive and negative range. Due to the internal bias, the positive

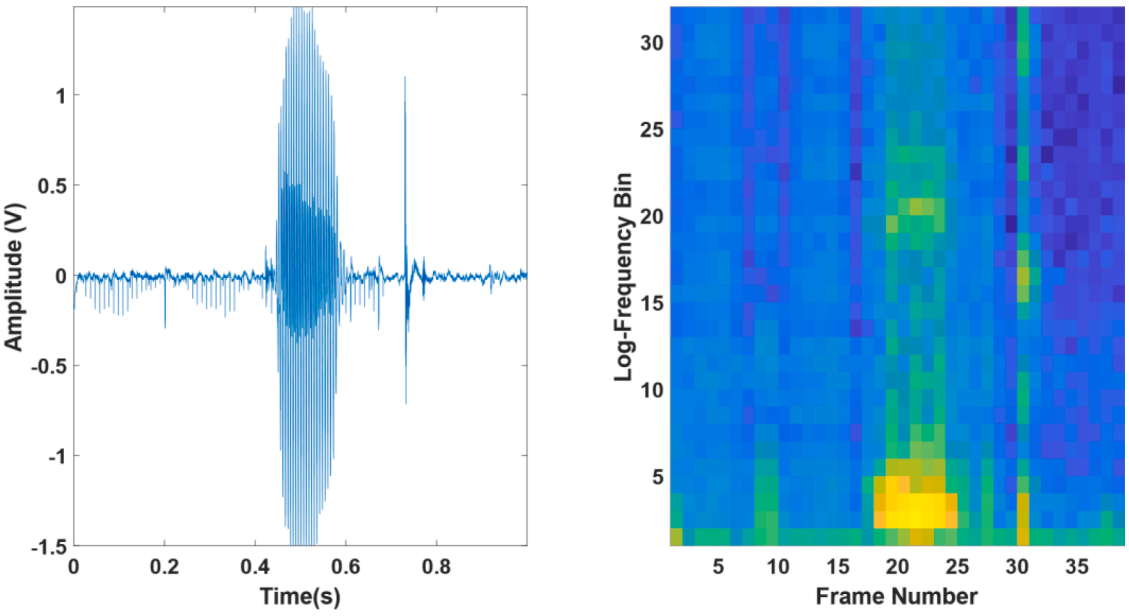



**Fig. 8.** Real-time speech signal for the digit 'FIVE' (b) Spectrogram.

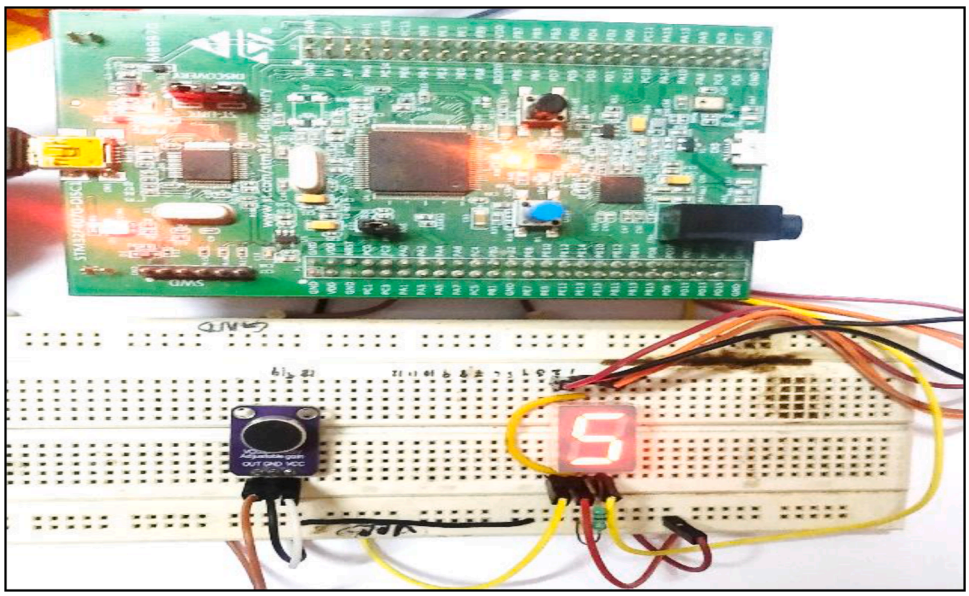


**Fig. 9.** Speech Digit Classification Hardware Setup.

half of the signal lies in the range of 1.5–3 Volts, and the negative half lies within 1.5–0 V. The microphone sensor has a gain adjustment resistor and a low-pass passive filter to filter out the dc component. The output of MAX4466 IC is fed directly to the ADC pin (PA0) of the STM32F407 discovery board.

### 5.2. Data acquisition using DMA

The STM32F407VG has a maximum operating clock frequency of 168 MHz. Using the registers under the Reset and Clock Control (RCC) peripheral, the system clock and the clock for various peripherals and ports can be configured. The clock for the ADC peripheral is set to 1 MHz, and the resolution is selected as 8bits to sample the analog input at 8000 Hz. The system clock is fixed at 128 MHz. Under the current clock and resolution settings, the ADC consumes about 120 clock cycles for one sample conversion. Based on this specification, sampling time and frequency are determined by the below equations,

$$\begin{aligned} Sampling\text{time}\left(\mathrm{T_s}\right) &= \frac{120}{1\times 10^6} = 120\mu s(1\text{ADCsample}) \\ Sampling\text{frequency}\mathrm{F_s} &= \frac{1}{\mathrm{T_s}} = 8333Hz \approx 8KHz \end{aligned} \tag{13}$$

DMA is configured to generate an interrupt every 1 s or 8000 samples to achieve continuous data acquisition while the CPU is processing data. Stream0 of the DMA1 peripheral is used in interrupt mode (half and full complete) to collect the analog data from the PA0 pin and to process the data further. Clock control for DMA1 comes under RCC AHB1 peripheral. The 8-bit ADC data from DMA is filled in a buffer size of 16,000, representing 2 s of the signal since the sampling frequency of the ADC peripheral is approximately 8000 Hz. The DMA interrupts are configured so that when the first half of the buffer, with 0–7999 samples are filled, it triggers the half-complete interrupt and the full complete interrupt is triggered when the second half with 8000–15,999 samples are filled. The first half of the buffer is referred to as 'In0' and the second half is named 'In1'. Once the ADC fills 'In0', a half-complete interrupt initiates the processing of the first half, and the 'In1' gets filled creating a cycle of events as shown in Fig. 7.

The data acquired using the DMA is then used to compute the spectrogram. CMSIS-DSP library is used to calculate FFT to generate the spectrogram. Fig. 8 shows the sound signal acquired for digit 'FIVE' and its log scaled spectrogram.

**Table 2**
Speech digit model parameters, memory consumption, and computation time for the quantized model.

| Layer number | Layer type | Filter shape | Output shape | Operations | Runtime (in ms) |
|---|---|---|---|---|---|
| Layer 1 | Convolution | 3 × 3 × 1 × 12 (0.11 KB) | 32 × 39 × 12 (14.63 KB) | 269.6 K | 14 |
| Layer 2 | RELU | N.A. | N.A. | 14.9 K | 0.6 |
| Layer 3 | Max pooling | N.A. | 16 × 20 × 12 (3.75 KB) | 34.6 K | 1 |
| Layer 4 | Convolution | 3 × 3 × 12 × 24 (2.53 KB) | 16 × 20 × 24 (7.5 KB) | 1.7 M | 20 |
| Layer 5 | RELU | N.A. | N.A. | 7.7 K | 0.1 |
| Layer 6 | Max pooling | N.A. | 8 × 10 × 24 (1.88 KB) | 17.3 K | 1 |
| Layer 7 | Convolution | 3 × 3 × 24 × 48 (10.13 KB) | 8 × 10 × 48 (3.75 KB) | 1.7 M | 18 |
| Layer 8 | RELU | N.A. | N.A. | 3.8 K | 0.06 |
| Layer 9 | Max pooling | N.A. | 8 × 6 × 48 (2.25 KB) | 11.5 K | 1 |
| Layer 10 | Fully-connected | 12 × 2304 (27.0 KB) | 12 | 55.3 K | 0.4 |
| Total | | 40KB | 34KB | 3.8 M | 56.16 ms |

### 5.3. CMSIS-NN based implementation

After the framework computes the *q7* format equivalent for the parameters of the CNN by using the algorithms mentioned, they are loaded into the microcontroller's memory. Open-sourced CMSIS-NN library functions are used for convolutions, ReLU, max-pooling, and fully connected layers. During real-time implementation, the spectrogram acts as the input for the first layer and needs to be converted into *q7* format for faster computations. For each layer, among other input parameters, finding the re-quantization factor for convolution functions is crucial to obtain the inference.

The proposed voice digit classification system's hardware setup is shown in Fig. 9 and it is to be evaluated on three aspects: memory consumption, computation time, and the model's recognition accuracy [27]. The learning parameters of the CNN model are consuming $\approx 40kB$

**Table 3**
Framework comparison by memory requirements.

| Framework | Sine model memory | OCR model memory | Speech digit model memory | Key-word spotter memory |
|---|---|---|---|---|
| X-Cube AI | 12.3 kB flash + 2.01kB RAM | 119.38kB flash + 24.42kB RAM | 102.54kB flash + 19.07kB RAM | 523.6kB flash + 80.09kB RAM |
| Proposed Framework | 7.41kB flash + 2.32kB RAM | 82.32kB flash + 38.34kB RAM | 61.71kB flash + 39.66kB RAM | 503.18kB flash + 71.7 kB RAM |

**Table 4**
Relation between clock frequency and current consumption for STM32F407 discovery board.

| Clock frequency (MHz) | Current consumed (mA) | Sine model inference time (ms) | OCR model inference time (ms) | Speech digit model inference time (ms) | Keyword spotter model inference time (ms) |
|---|---|---|---|---|---|
| 168 | 87 | 0.018 | 46 | 44 | 82 |
| 144 | 67 | 0.021 | 54 | 51 | 93 |
| 90 | 56 | 0.032 | 86 | 81 | 145 |
| 60 | 44 | 0.049 | 129 | 122 | 218 |

of flash memory which is 4x times less than the *float32* (see Table 1). The buffers used for STFT, the CNN layers, the input data buffer and other variables for the data collection and inference system takes around *110kB* in RAM. For computation, the system takes ≈ 70*ms* per inference (*56.*16 ms + ∼13 ms for spectrogram calculation) which was obtained by enabling the instruction and data cache. This can be equated to 14 inferences per second (see Table 2).

The system's real-time recognition accuracy is measured by 500 inputs belonging to different gender and age groups under a normal home environment with usual noises. From the confusion matrix for the quantized model, the overall accuracy was determined to be ≈85%. It can be improved in different ways like increasing the number of CNN layers, microphone with automatic gain control, etc. The system consumes approximately ≈26 mA at 128 MHz, and if we consider a normal Ni-Cd AA battery of 2400mAh it can run up to ≈92 h. Once the inference process is completed, the classified digit is displayed in a 7-segment display interfaced with the microcontroller.

## 6. Framework evaluation

This section compares the performance of the models deployed using the proposed framework with already existing and established frameworks like TensorFlow lite for microcontrollers and X-Cube-AI by STMicroelectronics. TensorFlow lite is a generalised framework and X-Cube-AI is targeted for STM32 based microcontrollers alone. Among these two softwares, it was proved by [28], that the performance of X-Cube-AI is better than TensorFlow lite for microcontrollers in terms of both memory and computation time. So, we have compared our framework's performance with the more efficient X-Cube-AI platform. Apart from the voice digit recognition model, the framework is tested with other tiny models including Optical Character Recognition (OCR) for 0–9 numbers and A-Z alphabets, time-series prediction of the sine wave and keyword spotter (similar architecture to [28],). As the proposed framework aims to provide a solution for deploying models in a memory efficient way, the comparison is provided by keeping the size requirement as the metric. From the results obtained as seen in Table 3, the flash and RAM memory used by the proposed framework is lesser than the requirement by X-Cube-AI.

Apart from the memory reduction offered by the framework, another advantage of using a quantized model is saving energy consumed by the device. For standalone embedded systems, it is important that the device consumes lesser energy. This can be achieved by disabling the Floating-Point Unit (FPU) during the inference process. Another method to save significant amount of energy is by reducing the clock frequency. Using a quantized model provides reduction in computation time, which in turn would allow us to use a lesser clock frequency. A comparison was done to observe how the current consumption varies with core clock along with the inference time taken (in milliseconds) for different models as seen in Table 4. Based on the data obtained, we can reduce the clock frequency from maximum of 168 MHz to 60 MHz, halving the energy requirement and still complete the inference in a short period. The current consumption data was obtained from the production datasheet [29].

## 7. Conclusion

The framework proposed is used to deploy a trained CNN model in a Cortex-M4 microcontroller, automating the quantization and inference code generation stages. This work enables a more accessible option for researchers and developers to deploy their Neural Networks in the microcontroller for various applications at low cost. The framework is evaluated with several models and the code generated using the proposed method is compared with existing tools like X-Cube-AI. The results show that the proposed method consumes lesser memory compared with X-Cube-AI and also maintains its accuracy similar with the floating-point model. In addition, the proposed scheme has been implemented in less than 25$ which promises the reality of various on-device AI applications. The framework can be found in the GitHub repository (link for framework).,

## Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

No data was used for the research described in the article.

## References

[1] S. Branco, A.G. Ferreira, J. Cabral, Machine Learning in resource-scarce embedded systems, FPGAs, and end-devices: a survey, Electronics 2019 8 (11) (2019) 1289.
[2] G. David, O. Jon, P. Javier, G. Fernando, M. Patxi, Data-driven fault diagnosis for electric drives: a review, Sensors (2021).
[3] P.E. Novac, A. Russo, B. Miramond, A. Pegatoquet, F. Verdier, A. Castagnetti, Toward unsupervised human activity recognition on microcontroller units, in: Proceedings of the 2020 23rd Euro micro Conference on Digital System Design (DSD), Kranj, Slovenia, 26–28 August 2020, 2020, pp. 542–550.
[4] P. Juan, L. Duran, Daniel Gutierrez-Galan, R. Antonio, L. Alejandro, J. Angel, Wildlife monitoring on the edge: a performance evaluation of embedded neural networks on microcontrollers for animal behavior classification, Sensors (2021).
[5] D. Palossi, A. Loquercio, F. Conti, E. Flamand, D. Scaramuzza, L. Benini, A 64-mw dnn-based visual navigation engine for autonomous nano-drones, IEEE Internet of Things J. 6 (5) (2019) 8357–8371.
[6] Y. Zhang, N. Suda, L. Lai, V. Chandra, Hello Edge: keyword Spotting on Microcontrollers, arXiv 2018 (2018) arXiv:1711.07128.
[7] P.P. Ray, A review on TinyML: state-of-the-art and prospects, J. King Saud University-Comput. Inf. Sci. (2021).
[8] Swapnil Sayan Saha, Sandeep Singh Sandha, Mani Srivastava, Machine learning for microcontroller-class hardware – a review, arXiv (2022) arXiv:2205.14550.
[9] R. Vasanthakumar, K.R. Chandra, V. Ashok, N.G. Raj, Accurate and efficient fixed-point inference for deep neural networks, in: IEEE International Conference on Image Processing. Greece, 2018, p. 2018.
[10] R. Goyal, J. Vanschoren, V. van Acht, S. Nijssen, Fixed-point quantization of convolutional neural networks for quantized inference on embedded platforms, arXiv 2021 (2021) arXiv:2102.02147.

[11] A. Rodriguez, E. Segal, E. Meiri, E. Fomenko, Lower numerical precision deep learning inference and training, Intel White Paper (2018) 2018.
[12] C. Gianmarco, A. Renzo, C. Lukas, F. Elisabetta, Sound event detection with Binary neural networks on tightly power-constrained IoT devices, in: Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED '20), Association for Computing Machinery, New York, NY, USA, 2020, pp. 19–24, https://doi.org/10.1145/3370748.3406588.
[13] P.E. Novac, G.B. Hacene, A. Pegatoquet, B. Miramond, V. Gripon, Quantization and deployment of deep neural networks on microcontrollers, Sensors 2021 (2021).
[14] R. David, J. Duke, A. Jain, Vijay Janapa Reddi, TensorFlow lite micro: embedded machine learning on TinyML systems, arXiv (2021) 2021, arXiv:2010.08678.
[15] X-cube-ai: ai, Expansion pack for stm32cubemx - stmicroelectronics, [Online] (2022). Available, https://www.st.com/en/embedded-software/x-cube-ai.html.
[16] Janakiram M.S.V. (2020). How TinyML makes artificial intelligence ubiquitous. Available online: https://www.forbes.com/sites/janakirammsv/2020/11/03/how-tinyml-makes-artificial-intelligence-ubiquitous/?sh=464ecc607622 (Accessed on 1 August 2021).
[17] S.R. Jain, A. Gural, M. Wu, C.H. Dick, Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks, in: Proceedings of the 3rd MLSys Conference, Austin, TX, USA, 2020, p. 2020.
[18] A. Shawahna, S.M. Sait, A. El-Maleh, I. Ahmad, FxP-QNet: a post-training quantizer for the design of mixed low-precision DNNs with dynamic fixed-point representation, IEEE Access 10 (2022) 30202–30231, https://doi.org/10.1109/ACCESS.2022.3157893, 2022.
[19] M. Dorfler, T. Grill, Inside the spectrogram: convolutional neural networks in audio processing, in: International Conference on Sampling Theory and Applications (SAMPTA), July 2017, Tallinn, Estonia, 2017.
[20] P. Warden, Speech commands: a public dataset for single-word speech recognition. https://arxiv.org/abs/1804.03209.
[21] J.T. Huang, J. Li, Y. Gong, An analysis of convolutional neural networks for speech recognition, in: 2015 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2015, South Brisbane, Queensland, Australia, 2015, pp. 4989–4993. April 2015.
[22] A. Sehgal, N. Kehtarnavaz, A convolutional neural network smartphone app for real-time voice activity detection, IEEE Access 6 (2018) 9017–9026, 2018.
[23] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, arXiv preprint (2014) arXiv:1412.6980, 2014.
[24] S. Gupta, A. Agrawal, K. Gopalakrishnan, P. Narayanan, Deep learning with limited numerical precision, arXiv 2015, arXiv:1502.02551, . (2015).
[25] TensorFlow Lite for Microcontrollers, Available online: https://www.tensorflow.org/lite/microcontrollers. Accessed on 29 July 2020.
[26] Electret microphone with amplifier IC MAX4466, Available online: Generic Electret Microphone Amplifier MAX4466 (2020) (accessed on 03 March.
[27] A.B. Nassif, I. Shahin, Imtinan B Attili, Mohammad Azzeh, Speech recognition using deep neural networks: a systematic review, IEEE Access 7 (2019) 19143–19165, https://doi.org/10.1109/ACCESS.2019.2896880.
[28] U.Abid Osman, L. Gemma, M. Perotto, D. Brunelli, TinyML platforms benchmarking, in: International Conference on Applications in Electronics Pervading Industry, Environment and Society, Springer, 2022.
[29] Production datasheet for STM32F407xx, Available online: https://www.st.com/resource/en/datasheet/dm00037051.pdf. Acessed on 10 July 2022.



**Sailesh M** is currently pursuing B.Tech degree in Vellore Institute of Technology, Vellore. His-research interests are focused on embedded systems, signal processing and machine learning.



**K Selvakumar**, currently working as Senior Assistant Professor at Vellore Institute of Technology in India. He received his ME-(Embedded Systems) degree from College of Engineering, Anna University, Chennai, India in 2008. He was awarded Ph.D degree in the area of embedded vision systems from Anna University, in 2015. His-research interests include signal processing, embedded vision systems, and on-device AI. He has published more than 10 research papers and actively working on industry sponsored consultancy works.



**N. Narayanan Prasanth** is an Associate Professor at School of Computer Science and Engineering, VIT Vellore India. He received his B.Tech IT from Pondicherry University, M.E CSE from Anna University and Ph.D CSE from M.S University Tirunelveli. His-research interest includes Network Switch Scheduling, Switching architecture and High Performance Computing. He has published more than 40 papers in various conferences and journals. He is a life member of ISTE and member of IEEE.